Assignment 8: Time Series Analysis

Sarah Hall

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

Directions

1. Rename this file `<FirstLast>_A08_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:

- Check your working directory
- Load the tidyverse, lubridate, zoo, and trend packages
- Set your ggplot theme

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

```r
library(lubridate)
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```
library(trend)
library(here)
```

```
## here() starts at /home/guest/EDE_Fall2025
```

```
#check working directory
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
#create mytheme, make text size bigger, axis text black, and legend on the top
mytheme <- theme_classic(base_size = 14)
  theme(axis.text = element_text(color = "black"),
        legend.position = "top")
```

```
## List of 2
##  $ axis.text      :List of 11
##   ..$ family       : NULL
##   ..$ face         : NULL
##   ..$ colour       : chr "black"
##   ..$ size         : NULL
##   ..$ hjust        : NULL
##   ..$ vjust        : NULL
##   ..$ angle        : NULL
##   ..$ lineheight   : NULL
##   ..$ margin       : NULL
##   ..$ debug        : NULL
##   ..$ inherit.blank: logi FALSE
##   ..- attr(*, "class")= chr [1:2] "element_text" "element"
##  $ legend.position: chr "top"
##  - attr(*, "class")= chr [1:2] "theme" "gg"
##  - attr(*, "complete")= logi FALSE
##  - attr(*, "validate")= logi TRUE
```

```
#set mytheme to standard theme
theme_set(mytheme)
```

2. Import the ten datasets from the Ozone_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named `GaringerOzone` of 3589 observation and 20 variables.

```r
#1

#import all 10 datasets
EPA_2010 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2010_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2011 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2011_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2012 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2012_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2013 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2013_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2014 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2014_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2015 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2015_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2016 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2016_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2017 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2017_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2018 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2018_raw.csv'),
                     stringsAsFactors = TRUE)
EPA_2019 <- read.csv(here('Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2019_raw.csv'),
                     stringsAsFactors = TRUE)

#combine using rbind
GaringerOzone <- rbind(EPA_2010, EPA_2011, EPA_2012, EPA_2013, EPA_2014,
                       EPA_2015, EPA_2016, EPA_2017, EPA_2018, EPA_2019)
```

Wrangle

3. Set your date column as a date class.

4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY_AQI_VALUE.

5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to "Date".

6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```r
# 3
GaringerOzone$Date <- as.Date(GaringerOzone$Date, format = "%m/%d/%Y")

# 4
GaringerOzone_Filtered <-
  GaringerOzone %>%
    select(Date, Daily.Max.8.hour.Ozone.Concentration, DAILY_AQI_VALUE)

# 5
```

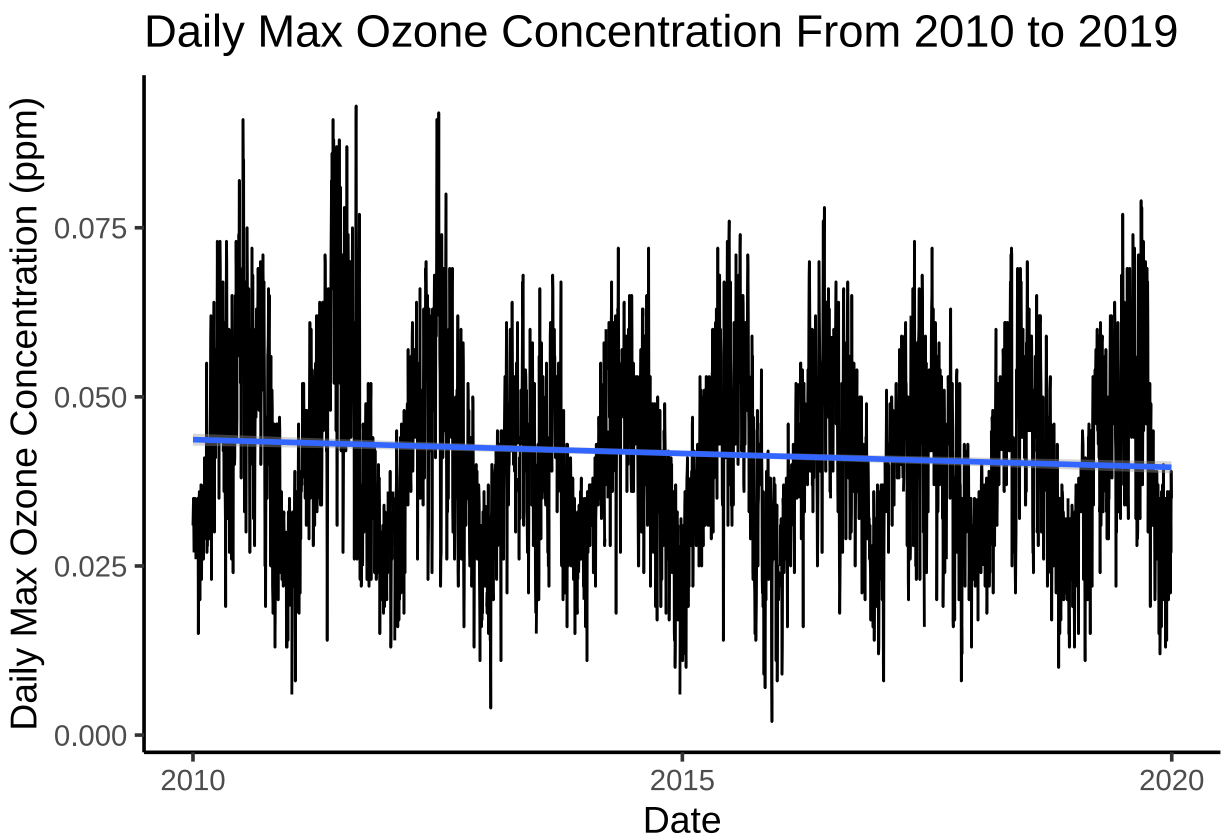
```r
Days <-
  as.data.frame(seq(from=as.Date("2010-01-01"),
                    to=as.Date("2019-12-31"),
                    by="day"))
#rename column to Date
colnames(Days) <- "Date"

# 6
GaringerOzone1 <-
  left_join(Days, GaringerOzone_Filtered, by = "Date")
```

Visualize

7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

```r
#7
GaringerOzone1 %>%
  ggplot(
    aes(x = Date,
        y = Daily.Max.8.hour.Ozone.Concentration)) +
  geom_line() +
  labs(
    y = "Daily Max Ozone Concentration (ppm)",
    title = "Daily Max Ozone Concentration From 2010 to 2019") +
  geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```



Answer: The plot suggests a strong seasonal trend, as well as a slight decline in ozone concentration over the time period.

Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

```
#8
GaringerOzone1 <-
  GaringerOzone1 %>%
  mutate(Daily.Max.8.hour.Ozone.Concentration = zoo::na.approx(Daily.Max.8.hour.Ozone.Concentration))

#double check for NAs
summary(GaringerOzone1)
```

```
##       Date            Daily.Max.8.hour.Ozone.Concentration DAILY_AQI_VALUE
##  Min.   :2010-01-01   Min.   :0.00200                      Min.   :  2.00
##  1st Qu.:2012-07-01   1st Qu.:0.03200                      1st Qu.: 30.00
##  Median :2014-12-31   Median :0.04100                      Median : 38.00
##  Mean   :2014-12-31   Mean   :0.04151                      Mean   : 41.57
##  3rd Qu.:2017-07-01   3rd Qu.:0.05100                      3rd Qu.: 47.00
```

```
##  Max.   :2019-12-31   Max.   :0.09300                       Max.   :169.00
##                                                              NA's   :63
```

Answer: Linear interpolation fills in ozone data by drawing.a straight line between the previous and next measurement, this appears to be a good fit for the data. Piecewise constant would create abrupt jumps between data since missing data is assumed equal to the nearest measurement. While spline is best for curved continous data, and could also create innacurate interploted points.

9. Create a new data frame called `GaringerOzone.monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

```r
#9
GaringerOzoneMonthly <-
  GaringerOzone1 %>%
  mutate(
    Year = year(Date),
    Month = month(Date)) %>%
  group_by(Year, Month) %>%
  summarise(Monthly_Mean_Ozone = mean(Daily.Max.8.hour.Ozone.Concentration))
```

```
## `summarise()` has grouped output by 'Year'. You can override using the
## `.groups` argument.
```

```r
#create new Date column
GaringerOzoneMonthly <-
  GaringerOzoneMonthly %>%
  mutate(Date = my(paste0(Month, "-", Year)))
```

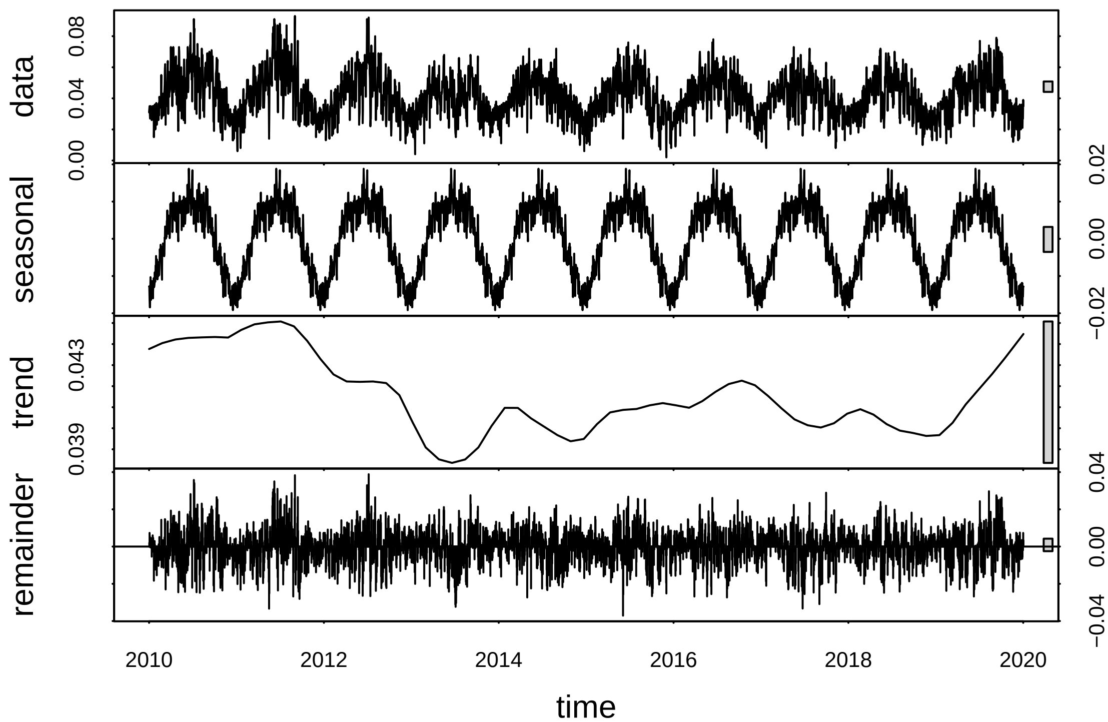
10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

```r
#10
#define first day and first year values
f_day <- yday(first(GaringerOzone1$Date))
f_year <- year(first(GaringerOzone1$Date))

#daily time series
GaringerOzone.daily.ts <-
  ts(GaringerOzone1$Daily.Max.8.hour.Ozone.Concentration,
     start=c(f_year, f_day),
     frequency = 365)

#define first month and first year values
f_month <- month(first(GaringerOzoneMonthly$Date))
f_year2 <- year(first(GaringerOzoneMonthly$Date))

#monthly time series
```

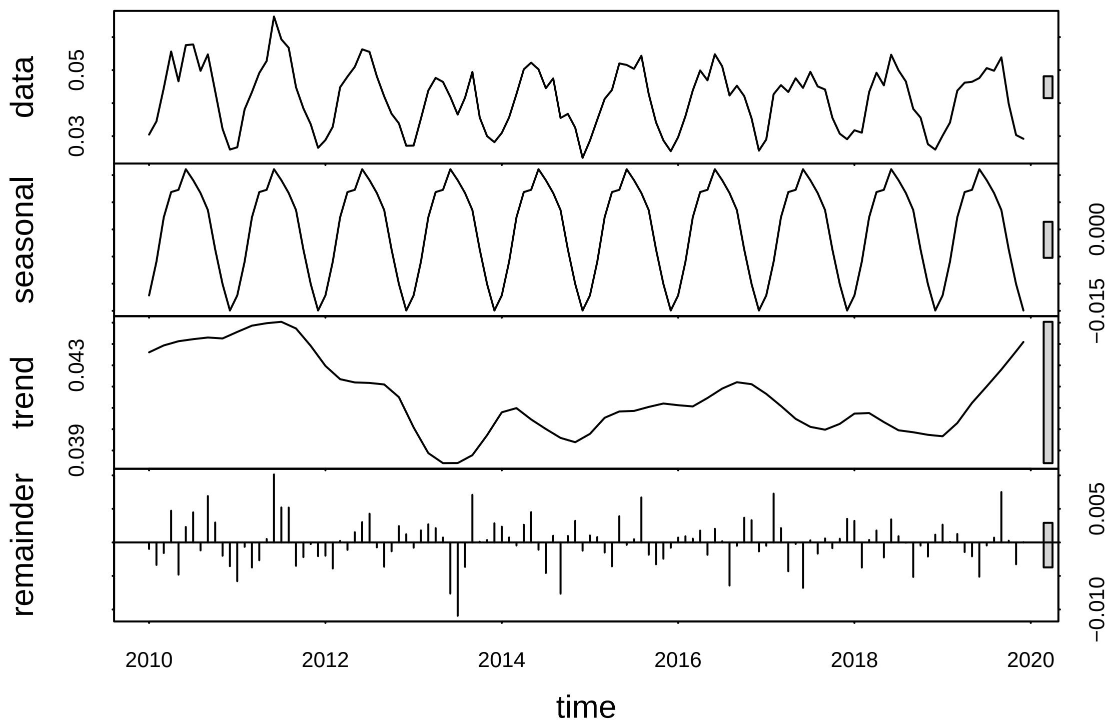
```
GaringerOzone.monthly.ts <-
  ts(GaringerOzoneMonthly$Monthly_Mean_Ozone,
     start = c(f_year2, f_month),
     frequency = 12)
```

11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

```
#11
#daily
garinger_ozone_daily_decomp <- stl(GaringerOzone.daily.ts,
                                   s.window = "periodic")
plot(garinger_ozone_daily_decomp)
```




```
#monthly
garinger_ozone_monthly_decomp <- stl(GaringerOzone.monthly.ts,
                                     s.window = "periodic")
plot(garinger_ozone_monthly_decomp)
```



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

```
#12

garinger_ozone_monthly_trend <- Kendall::SeasonalMannKendall(GaringerOzone.monthly.ts)
garinger_ozone_monthly_trend
```

```
## tau = -0.143, 2-sided pvalue =0.046724
```

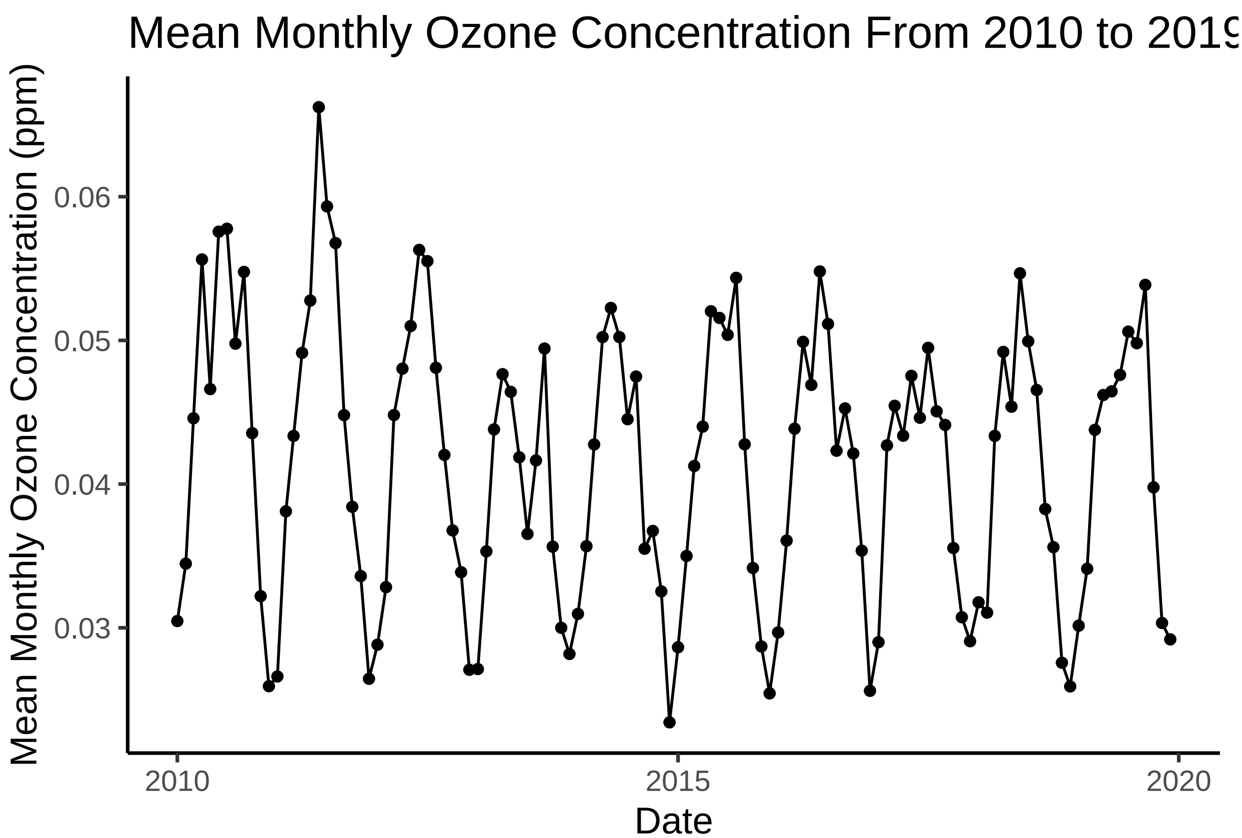
```
summary(garinger_ozone_monthly_trend)
```

```
## Score =  -77 , Var(Score) = 1499
## denominator =  539.4972
## tau = -0.143, 2-sided pvalue =0.046724
```

Answer: The seasonal Mann-Kendall trend analysis is most appropriate because there is a clear seasonal trend in the monthly ozone data series.

13. Create a plot depicting mean monthly ozone concentrations over time, with both a geom_point and a geom_line layer. Edit your axis labels accordingly.

```r
# 13
GaringerOzoneMonthly %>%
  ggplot(
    aes(
      x = Date,
      y = Monthly_Mean_Ozone))+
  geom_point()+
  geom_line()+
  labs(
    x = "Date",
    y = "Mean Monthly Ozone Concentration (ppm)",
    title = "Mean Monthly Ozone Concentration From 2010 to 2019"
  )
```



14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation.

    Answer:

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the EnoDischarge on the lesson Rmd file.

16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

```r
#15
#extract the components of each series into a datarame
ozone_monthly_compon <-
  as.data.frame(garinger_ozone_monthly_decomp$time.series[, 1:3])

#remove seasonal
Monthly_Ozone_Non_Seas <-
  GaringerOzone.monthly.ts - ozone_monthly_compon[,1]

#16
Ozone_Monthly_noseas_trend <- Kendall::MannKendall(Monthly_Ozone_Non_Seas)
Ozone_Monthly_noseas_trend
```

```
## tau = -0.165, 2-sided pvalue =0.0075402
```

```r
summary(Ozone_Monthly_noseas_trend)
```

```
## Score =  -1179 , Var(Score) = 194365.7
## denominator =  7139.5
## tau = -0.165, 2-sided pvalue =0.0075402
```

Answer: